

gemini.google.com

Google Gemini

9–13 minutes

Para unificar o motor de raspagem do eRepublik com o gerador de temas aleatórios da Wikipedia e garantir que o sistema rode de forma contínua em um servidor de **hospedagem gratuita**, a melhor abordagem é converter a lógica anterior de Bash para **Python (app.py)**.

A maioria dos servidores gratuitos (como Render, Railway ou PythonAnywhere) não permite a execução de scripts Bash que interagem com o terminal via `tput`, mas oferecem suporte total a Python com bibliotecas de automação e raspagem de dados.

Abaixo, apresento a arquitetura completa do sistema convertida para um servidor web em Python que gera e atualiza os arquivos HTML em tempo real, rodando o loop em background a cada segundo.

Estrutura do Sistema em Python (app.py)

Este script utiliza `BeautifulSoup` para extrair os dados reais (incluindo a Região Atual e a lista de Amigos/Friends que você solicitou), realiza a rolagem dos 7 dados para o ID e mantém um loop infinito atualizando o banco de dados offline.

```
import os
import time
import random
```

```
import threading
import requests
from bs4 import BeautifulSoup
from flask import Flask, render_template_string

app = Flask(__name__)

# Configurações de Pastas
DIR_SITE = "./site_offline"
os.makedirs(DIR_SITE, exist_ok=True)

# Estado Global do Jogo (Memória)
db_npcs = []
db_temas = []
ids_processados = set()
ponteiro_npc = 0
limite_max_npcs = 101

# HTML Base da Página Principal (Dashboard do RPG)
DASHBOARD_HTML = """
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>BBS RPG Engine - Painel Online</
title>
    <meta http-equiv="refresh" content="3">
<style>
    body { background: #111; color: #0f0;
font-family: monospace; padding: 20px; }
    .container { display: flex; gap: 20px;
```

```
}
    .box { border: 1px solid #0f0; padding:
15px; width: 50%; height: 500px; overflow-y:
scroll; }
    a { color: #0af; text-decoration: none;
}
    a:hover { text-decoration: underline; }
    h2 { border-bottom: 1px solid #0f0;
padding-bottom: 5px; }
</style>
</head>
<body>
    <h1>[BBS ENGINE] - Monitor de Sistema em
Tempo Real</h1>
    <p><b>Status:</b> Servidor Ativo | <b>Ciclo
Atual de NPCs:</b> Indexado {{ npcs_count
}}/101</p>
    <hr>
    <div class="container">
        <div class="box">
            <h2>[Wikipedia] Banco de Temas
Recentes (1s)</h2>
            <ul>
                {% for tema in temas[::-1][:20]
%}
                    <li><a href="{ {{ tema.link
}}" target="_blank">{{ tema.titulo }}</a></li>
                {% endfor %}
            </ul>
        </div>
        <div class="box">
            <h2>[eRepublik] Monitor de Cidadãos
```

```
/ NPCs</h2>
    <ul>
        {% for npc in npcs %}
            <li>
                <b>ID:</b> {{ npc.id }}
- <a href="/npc/{{ npc.id }}">{{ npc.nome }}</
a><br>
                <small>Loc: {{
npc.regiao }} | MU: {{ npc.mu }} | Partido: {{
npc.partido }}</small>
            </li>
        {% endfor %}
    </ul>
</div>
</div>
</body>
</html>
"""

def rolar_id_7_digitos():
    return "".join(str(random.randint(0, 9))
for _ in range(7))

def buscar_tema_wikipedia():
    """Puxa um tema aleatório da Wikipedia real
via link de Especial:Aleatória"""
    global db_temas
    headers = {"User-Agent": "Mozilla/5.0 (BBS
RPG Engine)"}
    try:
        res = requests.get("https://
pt.wikipedia.org/wiki/Especial:Aleat%C3%B3ria",
```

```
headers=headers, timeout=5)
    soup = BeautifulSoup(res.text,
'html.parser')
    titulo =
soup.find(id="firstHeading").text
    link = res.url
    db_temas.append({"titulo": titulo,
"link": link})
    except Exception:
        pass

def raspar_perfil_erepublik(id_cidadao):
    """Acesse o perfil online extraindo
informações verídicas do eRepublik"""
    url = f"https://www.erepublik.com/en/
citizen/profile/{id_cidadao}"
    headers = {"User-Agent": "Mozilla/5.0
(Windows NT 10.0; Win64; x64)"}

    try:
        res = requests.get(url,
headers=headers, timeout=5)
        if res.status_code != 200:
            return None

        soup = BeautifulSoup(res.text,
'html.parser')

        # Extração de Dados Verídicos usando
Seletores HTML do eRepublik
        nome =
soup.find(class_="citizen_name").text.strip()
```

```
if soup.find(class_="citizen_name") else
f"NPC_{id_cidadao}"

        # Região Atual
        regioao_elem =
soup.find(class_="citizen_region")
        regioao = regioao_elem.text.strip() if
regioao_elem else "Desconhecida"

        # Alinhamentos e Hiperlinks (MU,
Partido, Jornal)
        mu = soup.find(href=lambda x: x and
"group-show" in x)
        mu_nome = mu.text.strip() if mu else
"Nenhuma"

        partido = soup.find(href=lambda x: x
and "party" in x)
        partido_nome = partido.text.strip() if
partido else "Nenhum"

        jornal = soup.find(href=lambda x: x and
"newspaper" in x)
        jornal_nome = jornal.text.strip() if
jornal else "Nenhum"

        # Amigos (Friends) - Contagem ou lista
básica
        amigos_elem =
soup.find(class_="friends_count")
        amigos = amigos_elem.text.strip() if
amigos_elem else "0"
```

```
        return {
            "id": id_cidadao, "nome": nome,
"regiao": regiao,
            "mu": mu_nome, "partido":
partido_nome, "jornal": jornal_nome,
            "friends": amigos, "url": url
        }
    except Exception:
        return None

def loop_background_motor():
    """Loop principal que roda a cada 1 segundo
    atualizando dados e rotatividade"""
    global db_npcs, ids_processados,
ponteiro_npc

    while True:
        # 1. Puxa tema da Wikipedia a cada
segundo
        buscar_tema_wikipedia()

        # 2. Gerenciamento do eRepublik com
limite de 101 NPCs
        if len(db_npcs) < limite_max_npcs:
            # Fase de Captura: Cria novos
rolando dados
            novo_id = rolar_id_7_digitos()
            if novo_id not in ids_processados:
                dados_reais =
raspar_perfil_erepublik(novo_id)
                if dados_reais:
```

```
        db_npcs.append(dados_reais)

ids_processados.add(novo_id)
    else:
        # Fase de Atualização Contínua:
        Repassa desde o primeiro atualizando dados
        mutáveis

        if ponteiro_npc >= len(db_npcs):
            ponteiro_npc = 0 # Reinicia o
            ciclo para atualizar do começo

            npc_alvo = db_npcs[ponteiro_npc]
            dados_atualizados =
            raspar_perfil_erepublik(npc_alvo["id"])
            if dados_atualizados:
                # Atualiza mantendo as mudanças
                de MU, Partido ou Jornal ocorridas na
                plataforma real

                db_npcs[ponteiro_npc] =
                dados_atualizados

            ponteiro_npc += 1

        time.sleep(1) # Intervalo estrito de 1
        segundo

# Rotas Web para o Player interagir pelo
Navegador
@app.route('/')
def index():
    return
    render_template_string(DASHBOARD_HTML,
```

```
npcs=db_npcs, temas=db_temas,
npcs_count=len(db_npcs))

@app.route('/npc/<id_npc>')
def perfil_npc(id_npc):
    npc = next((item for item in db_npcs if
item["id"] == id_npc), None)
    if not npc:
        return "NPC não encontrado ou ainda não
processado no ciclo.", 404

    # Renderização da página individual do NPC
    contendo sua infraestrutura atualizada
    perfil_template = """
    <body style="background:#222; color:#fff;
font-family:monospace; padding:20px;">
        <h1 style="color:#0f0;">Perfil do
Cidadão: {{ npc.nome }}</h1>
        <p><b>ID Sorteado:</b> {{ npc.id }}</p>
        <p><b>Região Localizada:</b> {{
npc.regiao }}</p>
        <p><b>Total de Aliados (Friends):</b>
{{ npc.friends }}</p>
        <hr>
        <h3>Afiliações Atuais detectadas na
Camada Virtual:</h3>
        <ul>
            <li><b>Unidade Militar (MU):</b> {{
npc.mu }}</li>
            <li><b>Partido Político:</b> {{
npc.partido }}</li>
            <li><b>Imprensa / Jornal:</b> {{
```

```
npc.jornal }}</li>
    </ul>
    <br>
    <p><a href="/"
style="color:#0af;">[Voltar ao Painel Geral]</
a> | <a href="{{ npc.url }}" target="_blank"
style="color:#ff0;">[Ver Perfil Original
Online]</a></p>
    </body>
    ""
    return
render_template_string(perfil_template,
npc=np)

if __name__ == '__main__':
    # Dispara o motor de segundo em segundo em
    uma Thread separada do Servidor Web

threading.Thread(target=loop_background_motor,
daemon=True).start()
    # Porta padrão para servidores em Nuvem
    port = int(os.environ.get("PORT", 5000))
    app.run(host='0.0.0.0', port=port)
```

Onde Hospedar de Graça? (Servidores Suportados)

Como transformamos o script em uma aplicação web estruturada em **Python + Flask**, você pode utilizar os seguintes serviços que possuem **planos gratuitos permanentes (Hobby/Free Tiers)**:

1. **Render (render.com)**: É a alternativa gratuita mais popular.

Você vincula seu repositório do GitHub e ele faz o deploy automático do arquivo `app.py`.

2. **PythonAnywhere (pythonanywhere.com)**: Excelente para scripts Python que rodam tarefas em background. O plano gratuito oferece um subdomínio próprio para você acessar de onde quiser.
3. **Railway (railway.app)**: Oferece créditos gratuitos mensais para desenvolvedores executarem microsserviços e contêineres que necessitem rodar loops contínuos.

Arquivo extra necessário para a Hospedagem (`requirements.txt`)

Ao subir para o servidor escolhido, crie um arquivo chamado `requirements.txt` na mesma pasta para que a hospedagem saiba quais dependências instalar:

```
Flask==3.0.2
requests==2.31.0
beautifulsoup4==4.12.3
```